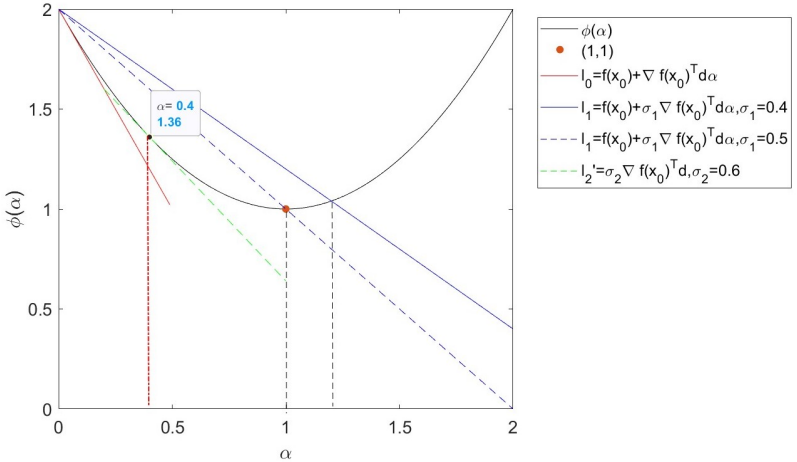
最优化理论与方法
(上机练习 2)
下降算法比较
(最速下降法、牛顿法和修正牛顿法)

上机练习 1 的说明

- 下载上机练习 1 作业 https://www.jianguoyun.com/p/DQB3F9cQpLmeDBiawaYGIAA
- 优秀作业：方宇清（matlab）、李春阳（advance_retreat）、雷龙浩（python）、李欣瑶、路程轶、唐梓杰（c++、实验报告）

上机练习 1 的说明

- wolfe-powell 条件 2 里的循环更新，$alpha = alpha/rho$ 会死循环
- 错误写法 $\alpha = \alpha + (\beta - \alpha)\rho^i$
- 调参没有变化



- 两个参数同时分析，meshgrid，实验结论应该有对应的数值结果支撑

下降算法

- 下降算法：$f(x_{k+1}) < f(x_k)$
- $x_{k+1} = x_k + \alpha_k d_k$

Algorithm 1 下降算法的一般步骤

1: 给定初始点 $x_0 \in \mathbb{R}^n$，精度 $\epsilon > 0$，$k := 0$
2: 若 $\|\nabla f(x_k)\| \leq \epsilon$，则停止，得解 x_k。否则转步 3
3: 确定下降方向 d_k
4: 确定步长 $\alpha_k > 0$，使得 $f(x_k + \alpha_k d_k) < f(x_k)$
5: 更新 $x_{k+1} = x_k + \alpha_k d_k$，$k := k + 1$，转步 2

最速下降法和牛顿法

Algorithm 2 最速下降法 (Steepest descent method)

Given $x_0 \in \mathbb{R}^n$, $\epsilon > 0$, $k := 0$
for $k = 0, 1, 2, \cdots$ **do**
    If $\|\nabla f(x_k)\| \leq \epsilon$, **stop**
    $$d_k = -\nabla f(x_k)$$
    line search for α_k
    $x_{k+1} = x_k + \alpha_k d_k$
    $k := k + 1$
end for

Algorithm 3 牛顿法 (Newton method)

Given $x_0 \in \mathbb{R}^n$, $\epsilon > 0$, $k := 0$
for $k = 0, 1, 2, \cdots$ **do**
    If $\|\nabla f(x_k)\| \leq \epsilon$, **stop**
    Compute
    $$\nabla^2 f(x_k) d_k = -\nabla f(x_k)$$
    line search for α_k
    $x_{k+1} = x_k + \alpha_k d_k$
    $k := k + 1$
end for

修正牛顿法

选择 $t_k > 0$ 使得 $\nabla^2 f(x_k) + t_k I$ 正定

Algorithm 4 修正牛顿法

Given $x_0 \in \mathbb{R}^n$, $\epsilon > 0$, $k := 0$
for $k = 0, 1, 2, \cdots$ **do**
    If $\|\nabla f(x_k)\| \leq \epsilon$, **stop**
    Compute

$$(\nabla^2 f(x_k) + t_k I) d_k = -\nabla f(x_k)$$

    line search for α_k
    $x_{k+1} = x_k + \alpha_k d_k$
    $k := k + 1$
end for

通过修正 Cholesky 分解计算 $\nabla^2 f(x_k) + t_k I = LDL^T$

令矩阵 $A := \nabla^2 f(x_k)$, ϵ 为机器精度，$\gamma = \max_{1\le i\le n} |a_{ii}|, \xi = \max_{i\neq j} |a_{ij}|$

Algorithm 5 修正 Cholesky 分解

```
给定对称矩阵 A = (a_ij), δ = ε max(γ(A) + ξ(A), 1) > 0, β = max(γ(A), ξ(A)/√(n^2 − 1), ε)^{1/2} > 0
for k = 1, 2, ··· , n do
    c_kk = a_kk
end for
for j = 1, 2, ··· , n do                                   ▷ 计算 L 的第 j 列
    l_jj = 1
    for s = 1, 2, ··· , j − 1 do
        l_js = c_js / d_s
    end for
    for i = j + 1, ··· , n do
        c_ij = a_ij − Σ_{s=1}^{j−1} l_js c_is
    end for
    θ_j = 0
    if j < n then
        θ_j = max_{{i:j<i≤n}} |c_ij|
    end if
    d_j = max{|c_jj|, (θ_j/β)^2, δ}
    if j < n then
        for i = j + 1, ··· , n do
            c_ii = c_ii − c_ij^2 / d_j
        end for
    end if
end for
```

上机练习 2-下降算法比较

1. 选择某种线性搜索方法编写最速下降算法、牛顿法和修正牛顿法
2. (选做）初始点取 $x_0 = (1.2, 1.2)$，求解 Rosenbrock 函数最小化问题：
$$\min \ f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2,$$
并打印出不同算法每一步迭代的步长。
3. 调好参数后，画出三个算法目标函数值随迭代次数的变化
4. 取 $x_0 = (-1.2, 1)$，分别采用黄金分割，Armijo 和 Wolfe-Powell 线搜索比较三个算法的迭代次数。
5. 尝试取不同初始点求解 Rastrigin 函数最小化问题
$$\min \ f(x) = 10n + \sum_{i=1}^{n}(x_i^2 - 10cos(2\pi x_i)), \ n = 6$$
6. 下载 LIBSVM 数据集 ‘a9a’，利用编写的算法求解逻辑回归模型(预测薪资水平)：
$$\min_x \frac{1}{m}\sum_{i=1}^{m}\log(1 + \exp(-b_i a_i^T x)) + \lambda\|x\|_2^2, \ \lambda = \frac{1}{100m}.$$

$$\min_x \frac{1}{m}\sum_{i=1}^{m}\log(1+\exp(-b_i a_i^T x)) + \lambda\|x\|_2^2,\ \ \lambda = \frac{1}{100m}.$$

其中 m 是样本数量。

- libsvm data sets: `https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/`
- matlab:
  - 从`https://www.csie.ntu.edu.tw/~cjlin/libsvm/`下载 libsvm 工具包
  - dataset = 'a9a.train'; [b,A] = libsvmread(dataset);
- python:
  - »from sklearn.datasets import load_svmlight_file
    »[A,b] = load_svmlight_file(dataset);
    »A = A.todense()
- CSV 格式：第一列是样本标签，每一行是一个数据的标签 b_i 和特征 a_i^T.
  - Matlab: data = csvread('a9a_train.csv');
  - python: data = pandas.read_csv('a9a_train.csv',header=None); data = data.values

请于3 月 29 日（周日）21:00前提交到

`https://send2me.cn/O6FuKjBg/RYuZ55mslP6JQA`

无约束优化问题测试集

- https://www.sfu.ca/~ssurjano/optimization.html
- https://github.com/ralna/CUTEst/wiki/
- Andrei, Neculai. "An Unconstrained Optimization Test Functions Collection." (2008).